

Rapport d'audit RGPD, sécurité et plan de remédiation — WellWork

Ce document présente l'audit complet réalisé sur la plateforme WellWork (code source et base de démonstration fournis), le plan d'action pour répondre aux exigences du grand compte et la gestion de crise suite à la fuite de données signalée sur le forum.

PARTIE A — Cartographie et conformité

1. Registre des traitements (article 30)

En analysant le code source dans `src/` et le script de peuplement `db/seed.js`, j'ai recensé 6 activités de traitement distinctes.

Réf	Finalité du traitement	Catégories de données traitées	Personnes concernées	Destinataires des données (Théoriques vs Constat dans le code)	Transferts hors UE	Durée de conservation (Cible légale vs Constat dans le code)
T-01	Gestion des comptes et authentification	Identité (nom, prénom, email, date de naissance, entreprise, rôle), hash de mot de passe (<code>passwordHash</code>), statut (<code>deleted</code> , <code>marketingOptIn</code>).	Salariés, RH des entreprises clientes, Coachs, Administrateurs.	Théoriques : L'utilisateur lui-même et l'équipe technique. Dans le code : N'importe quel utilisateur connecté peut lister tous les comptes via <code>GET /api/users</code> .	Aucun transfert hors UE documenté. Le serveur SMTP externe <code>mailprovider.io</code> configuré dans <code>config.js</code> doit faire l'objet d'un contrat de sous-traitance.	Cible : Durée du contrat de travail + 3 ans d'inactivité. Dans le code : Illimitée. <code>config.js</code> indique <code>retentionDays: null</code> et <code>DELETE /api/me</code> se contente d'un simple flag <code>deleted: true</code> .
T-02	Questionnaire de santé et bien-être (Données sensibles Art. 9)	Réponses médicales déclaratives : stress (1-10), antécédents médicaux, traitements en cours, sommeil, poids, taille, tabac (<code>seed.js</code> l.58-64).	Salariés utilisateurs.	Théoriques : Le salarié uniquement (et le coach référent s'il y consent). Dans le code : N'importe quel salarié peut voir les questionnaires des autres via <code>/api/users/:id</code> , et le RH peut tout exporter via <code>/api/exports/insurer</code> .	Aucun.	Cible : Durée d'activation du compte. Purge immédiate à la résiliation. Dans le code : Illimitée. Aucune suppression des questionnaires quand un compte est supprimé.
T-03	Messagerie et coaching individuel	Métadonnées (<code>from</code> , <code>to</code> , date <code>at</code>), contenu textuel libre des messages (<code>body</code>).	Salariés et Coachs.	Théoriques : Le salarié et son coach attitré. Dans le code : Stocké en texte clair dans <code>wellwork.json</code> sans aucun chiffrement.	Aucun.	Cible : Durée du suivi de coaching + 1 an. Dans le code : Illimitée.
T-04	Suivi des activités physiques	Historique des séances de sport (<code>sessionsSport</code> dans <code>db.js</code> l.13).	Salariés utilisateurs.	Théoriques : Salarié, coach, et statistiques agrégées sans nom pour le RH. Dans le code : Données non cloisonnées par entreprise.	Aucun.	Cible : Durée du programme. Dans le code : Illimitée.

Réf	Finalité du traitement	Catégories de données traitées	Personnes concernées	Destinataires des données (Théoriques vs Constat dans le code)	Transferts hors UE	Durée de conservation (Cible légale vs Constat dans le code)
T-05	Annuaire d'entreprise pour les RH	Nom, prénom, email professionnel, entreprise de rattachement.	Salariés de l'entreprise cliente.	Théoriques : Les RH de l'entreprise du salarié uniquement. Dans le code : Le RH d'ACME voit tous les salariés des entreprises concurrentes (Globex, Northwind) avec leurs hashes de passe (<code>data.js</code> l.31-34).	Aucun.	Cible : Durée du contrat commercial B2B avec l'entreprise. Dans le code : Illimitée.
T-06	Exports vers l'assureur partenaire	Profil complet des salariés et l'intégralité de leurs questionnaires de santé bruts.	Tous les salariés enregistrés.	Théoriques : Assureur partenaire. Dans le code : <code>G ET /api/exports/ insurer</code> envoie les données brutes de tout le monde sans filtre ni anonymisation.	Aucun.	Cible : Données uniquement agrégées et anonymisées si une convention existe. Dans le code : Illimitée. Export de données nominatives et médicales brutes.

2. Analyse de la base légale de chaque traitement et du statut des acteurs

A. Base légale de chaque traitement (Articles 6 et 9)

1. Gestion des comptes et annuaire (T-01 et T-05) :

- **Base légale : Exécution du contrat** (Art. 6.1.b) pour permettre aux salariés d'utiliser le service souscrit par leur entreprise, et **Intérêt légitime** (Art. 6.1.f) de l'employeur pour mettre à disposition cet avantage social.

2. Questionnaire de santé (T-02) :

- **Base légale :** Il s'agit de données de santé (Article 4.15), dont le traitement est formellement interdit par l'Article 9.1 du RGPD.
- La seule exception applicable ici est le **Consentement explicite** de l'utilisateur (Article 9.2.a).
- *Constat dans le code :* En testant la page `public/index.html` et la route `POST /api/questionnaires`, j'ai constaté qu'il n'y a aucune case à cocher ni mention d'information. Le salarié clique sur "Envoyer" et ses antécédents médicaux partent en base sans accord formel. Le traitement est donc **illégal en l'état**.

3. Messagerie et séances de sport (T-03 et T-04) :

- **Base légale : Exécution des CGU / contrat de service** (Art. 6.1.b) entre le salarié et WellWork.

4. Transmission à l'assureur (T-06) :

- **Base légale :** Ce transfert exige le **Consentement explicite et univoque** de chaque salarié (Art. 6.1.a et 9.2.a).
- *Constat dans le code :* Dans `routes/accounts.js` (l.32), le code insère d'office `consents: { thirdParty: true }` à l'inscription sans rien demander. Ce consentement automatique n'a aucune valeur juridique : l'export à

l'assureur est **totale**ment illégal.

B. Statut juridique des acteurs

- **WellWork (la start-up) — Responsable de Traitement indépendant (Art. 4.7) :**

C'est WellWork qui conçoit l'application, choisit les questions du questionnaire et gère la messagerie des coachs. WellWork doit impérativement faire écran entre les employeurs et les données de santé de leurs salariés.

- **L'entreprise cliente (Employeur / RH) — Responsable de Traitement distinct :**

L'employeur est responsable du traitement pour la transmission de la liste de ses effectifs. En revanche, **l'employeur n'a aucunement le droit d'être co-responsable sur les données de santé**. En droit du travail français et selon le RGPD, le secret médical interdit à l'employeur de voir la santé de ses employés en raison du lien de subordination. Il ne doit recevoir que des statistiques globales (ex : 65 % de participation).

- **Les coachs — Sous-traitants (Art. 28) :**

S'ils sont indépendants, ils agissent pour le compte de WellWork avec une obligation contractuelle de stricte confidentialité. S'ils sont salariés de WellWork, ils sont préposés sous son autorité directe (Art. 29).

- **L'assureur partenaire — Responsable de Traitement tiers :**

Il reçoit les données pour ses propres besoins d'assurance. Sans accord de transfert et sans consentement direct de chaque salarié, il n'a pas le droit de recevoir ces données.

3. Analyse d'impact sur la protection des données (AIPD)

A. Démonstration de l'obligation légale

D'après les lignes directrices européennes (WP248 / Article 35 du RGPD), une AIPD devient obligatoire dès qu'un traitement coche **au moins 2 critères sur les 9 de la grille CNIL** :

1. **Données sensibles (critère 4) :** Le questionnaire traite des données de santé (traitements, antécédents, stress).
2. **Personnes vulnérables (critère 7) :** Les salariés sont dans une situation de subordination vis-à-vis de leur employeur.
3. **Croisement ou combinaison d'ensembles de données (critère 6) :** L'outil recoupe l'identité professionnelle en entreprise, les échanges intimes avec les coachs et les réponses médicales.

Le projet cochant 3 critères sur 9 (les critères 4, 6 et 7), **l'AIPD est strictement obligatoire**.

B. Réalisation de l'AIPD sur le questionnaire de santé

1. **Description du fonctionnement :**

Le salarié saisit son stress et ses antécédents sur `public/index.html`. L'API stocke les réponses dans `db/wellwork.json` sans chiffrement. N'importe quel utilisateur peut ensuite les lire via `/api/users/:id`, et l'assureur les récupère via `/api/exports/insurer`.

2. **Proportionnalité et nécessité :**

- *Minimisation* : Non respectée. Demander les maladies chroniques (`antecedents`) et les médicaments (`traiteme nt`) est disproportionné pour un programme de bien-être en entreprise.
- *Consentement* : Non respecté. Aucun recueil explicite lors de la soumission.
- *Conservation* : Non respectée. Aucune purge automatique en place.

3. **Évaluation des risques sur les personnes (Grille d'impact) :**

Événement redouté	Faibles réelles dans le code	Impact pour les salariés	Vraisemblance	Gravité	Risque brut	Mesures de remédiation	Risque résiduel
Accès illégitime (Perte de confidentialité)	Faible sur <code>/api/users/:id</code> ; export assureur libre ; token admin codé en dur dans <code>seed.js</code> ; RCE sur le filtre (<code>db.js</code>).	Fuite des maladies, chantage, discrimination à l'embauche ou lors des promotions par les RH.	Maximale (4/4) (faibles ouvertures et exploitables sans authentification)	Critique (4/4) (données médicales intimes)	Critique	Contrôle d'accès strict (RBAC), cloisonnement par entreprise, suppression de l'export assureur brut, suppression de <code>new Function</code> .	Faible
Modification non désirée (Perte d'intégrité)	L'injection RCE permet de réécrire directement le fichier <code>wellwork.json</code> sur le disque.	Altération du suivi de santé, injection de fausses déclarations médicales.	Élevée (3/4)	Moyenne (2/4)	Élevé	Remplacer le fichier JSON par une base PostgreSQL sécurisée avec des transactions.	Faible
Disparition des données (Perte de disponibilité)	Tout repose sur un simple fichier <code>wellwork.json</code> réécrit avec <code>fs.writeFileSync</code> sans sauvegarde.	Perte des historiques médicaux des salariés.	Moyenne (2/4)	Moyenne (2/4)	Modéré	Sauvegardes automatiques chiffrées hors-site.	Négligeable

4. **Conclusion du DPO** : Le risque actuel est **inacceptable**. Ce questionnaire doit être suspendu tant que le cloisonnement et le recueil du consentement ne sont pas corrigés.

4. Constats de non-conformité RGPD

J'ai identifié 7 manquements majeurs dans l'application, numérotés de **NC-01** à **NC-07** pour correspondre directement au plan de remédiation :

Réf	Article RGPD	Manquement constaté	Preuve dans le code ou l'interface	Risque concret pour les salariés	Gravité
NC-01	Art. 6 et 9	Export illégal de données de santé brutes à l'assureur	<code>src/routes/data.js</code> (l.48-56) : <code>db.raw().users.map(...)</code> renvoie l'intégralité des 134 questionnaires médicaux à l'assureur. <code>src/routes/accounts.js</code> (l.32) force <code>thirdParty: true</code> sans choix de l'utilisateur.	Utilisation de données médicales à des fins d'assurance (hausse de cotisations, refus de couverture).	Critique
NC-02	Art. 5.1.f et 32	Absence de cloisonnement multi-tenant (secret médical violé)	<code>src/routes/data.js</code> (l.23-28) : <code>GET /api/users/:id</code> donne les questionnaires d'un collègue. <code>src/auth.js</code> (l.34) : le rôle <code>rh</code> a les mêmes accès que l'admin et voit les données des entreprises concurrentes.	Les RH et les collègues découvrent les antécédents médicaux des salariés. Risque direct de discrimination.	Critique
NC-03	Art. 5.1.c	Violation de la minimisation des données	<code>db/seed.js</code> (l.19-20 et l.58-64) et formulaire web : collecte de données médicales lourdes (<code>asthme</code> , <code>diabete type 2</code> , <code>anxiolytique</code> , <code>metformine</code>) disproportionnées pour du coaching bien-être.	Collecte intrusive d'informations médicales intimes sans justification valable.	Élevée
NC-04	Art. 17	Droit à l'effacement inefficace (pas de vraie suppression)	<code>src/routes/accounts.js</code> (l.62-65) : <code>DELETE /api/me</code> met juste <code>deleted: true</code> . Les questionnaires médicaux restent intacts en base.	Les données médicales sont conservées à vie malgré la demande formelle de suppression du salarié.	Élevée
NC-05	Art. 4.11 et 7	Consentement forcé à l'inscription	<code>src/routes/accounts.js</code> (l.27 et 32) : <code>marketingOptIn: true</code> et <code>consents (marketing: true, thirdParty: true)</code> sont insérés par défaut sans case à cocher.	L'utilisateur perd le contrôle de ses données ; démarchage et partages non voulus.	Élevée
NC-06	Art. 5.1.e	Absence de politique de purge des données	<code>src/config.js</code> (l.9) : <code>retentionDays: null, // pas de purge automatique pour l'instant</code> .	Conservation sans limite de temps, augmentant l'impact en cas de piratage.	Moyenne
NC-07	Art. 12 et 13	Défaut d'information légale des utilisateurs	<code>public/index.html</code> : aucune mention d'information, aucune politique de confidentialité, pas de contact DPO.	Les salariés ne savent pas qui traite leurs données ni comment exercer leurs droits.	Élevée

PARTIE B — Audit et sécurité

1. Constats de sécurité techniques

Chaque constat technique ci-dessous a été vérifié et testé sur l'instance locale (`localhost:3000`) :

SEC-01 — Exécution de code à distance (RCE) via `new Function`

- **Description** : Dans `src/db.js` (l.35), la fonction de recherche assemble le paramètre `filter` dans un `new Function('row', ...)`. Comme le développeur a mis un bloc `try/catch` qui retourne `false` en cas d'erreur, l'évaluation de code JavaScript arbitraire se fait de manière totalement invisible.

- **Preuve reproductible** : En exécutant cette commande curl dans mon terminal, j'appelle directement `child_process.execSync('id')` sur le serveur Node.js :

```
curl -s -G "http://localhost:3000/api/users" \
  --data-urlencode "token=MS4xLjE3MDk4MDAwMDAwMDA=" \
  --data-urlencode "filter=typeof row.constructor.constructor('return process')().getBuiltinModule('node:child_process').execSync('id').toString()===\'string\'" \
  | python3 -c "import json,sys; d=json.load(sys.stdin); print(len(d),\'lignes récupérées\')"
```

Le serveur répond avec succès (64 lignes) : la commande système a été exécutée par le processus Node.

- **Impact** : Prise de contrôle totale du serveur, lecture/écriture de `db/wellwork.json` et vol de tous les fichiers.
- **Score CVSS 3.1** : 9.9 Critique (`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H`)
- **OWASP / CWE** : A03:2021 (Injection) — CWE-94.

SEC-02 — Jeton admin statique dans le code source et tokens prédictibles

- **Description** : Dans `db/seed.js` (l.72), une session administrateur permanente est générée en encodant simplement en base64 `admin.id.1.1709800000000`, ce qui donne `MS4xLjE3MDk4MDAwMDAwMDA=`. De plus, `auth.js` (l.10) génère les jetons sans signature cryptographique ni secret. Ce jeton admin n'expire jamais.
- **Preuve reproductible** : Sans avoir besoin de mot de passe, j'interroge l'API avec ce jeton trouvé dans le dépôt :

```
curl -s "http://localhost:3000/api/me?token=MS4xLjE3MDk4MDAwMDAwMDA=" | python3 -m json.tool | head -1
0
```

L'API me renvoie directement le compte Administrateur (HTTP 200).

- **Impact** : N'importe qui consultant le dépôt GitHub ou récupérant ce jeton devient super-administrateur.
- **Score CVSS 3.1** : 9.8 Critique (`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`)
- **OWASP / CWE** : A07:2021 (Identification Failures) — CWE-798 (Hard-coded Credentials).

SEC-03 — Absence de rate-limiting et hachage trop faible (SHA-256 sans sel)

- **Description** : L'endpoint `POST /api/login` (`accounts.js` l.37-46) ne bloque pas les tentatives répétées. Les mots de passe sont hachés avec un simple SHA-256 sans sel (`db.js` l.26-28), ce qui permet de les casser instantanément avec des tables de hachage. De plus, `README.md` liste les mots de passe des comptes clés en clair (`Admin2024!`, `AcmeRh2024`, `coach123`).
- **Preuve reproductible** : J'ai envoyé 100 requêtes de connexion d'affilée en boucle :

```
for i in $(seq 1 100); do
  curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:3000/api/login \
    -H "content-type: application/json" -d '{"email":"admin@wellwork.example","password":"test"}'
done | sort | uniq -c
```

Le serveur a répondu 100 fois de suite avec un code 401 sans jamais bloquer ou ralentir l'attaquant.

- **Impact** : Attaques par force brute libres et cassage de mot de passe instantané.
- **Score CVSS 3.1** : 8.2 Élevé (`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N`)
- **OWASP / CWE** : A07:2021 — CWE-307, CWE-916.

SEC-04 — Exposition du hash de mot de passe dans les réponses API

- **Description :** La propriété `passwordHash` est retournée par 6 endpoints de l'API (`/api/me` , `/api/users/:id` , `/api/users` , etc.). Dans `seed.js` (l.47), les 60 salariés ne partagent que 5 mots de passe communs.
- **Preuve reproductible :**

```
curl -s http://localhost:3000/api/me -H "Authorization: Bearer MS4xLjE3MDk4MDAwMDAwMDA=" \
  | python3 -c "import json,sys; u=json.load(sys.stdin); print('passwordHash =', u.get('passwordHash'))"
```

Le hash apparaît directement dans la réponse JSON.

- **Impact :** Un salarié peut récupérer les hashes de tous ses collègues et les casser en quelques secondes.
- **Score CVSS 3.1 :** 8.1 Élevé (`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N`)
- **OWASP / CWE :** A01:2021 / A02:2021 — CWE-200.

SEC-05 — Défaut de cloisonnement multi-tenant et faille BOLA sur les questionnaires

- **Description :**
 1. `data.js` (l.23-28) : `GET /api/users/:id` renvoie le profil ET les questionnaires de santé de l'utilisateur demandé sans vérifier si le demandeur en est le propriétaire (faille BOLA).
 2. `data.js` (l.31-34) : `GET /api/users` retourne la base complète sans imposer le filtre sur l'entreprise du compte connecté.
 3. `auth.js` (l.34) : la vérification d'admin valide aussi le rôle `rh` , ce qui permet aux RH de lancer l'exportation globale `/api/exports/insurer` .
- **Preuve reproductible :** En me connectant avec le compte RH d'ACME, j'appelle l'export assureur :

```
curl -s -G "http://localhost:3000/api/exports/insurer" \
  --data-urlencode "token=Mt43LjE3OTYxNDYxODAxNTg=" \
  | python3 -c "import json,sys,collections; d=json.load(sys.stdin); print(len(d),'profils exposés'); print(collections.Counter(u.get('company') for u in d))"
```

Résultat : j'obtiens les 64 profils et 134 questionnaires de toutes les entreprises clientes (ACME, Northwind, Globex).

- **Impact :** Fuite totale des données de santé entre entreprises concurrentes.
- **Score CVSS 3.1 :** 8.8 Élevé (`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N`)
- **OWASP / CWE :** A01:2021 (Broken Access Control) — CWE-639, CWE-862.

SEC-06 — Mots de passe écrits en clair dans les fichiers de logs

- **Description :** Dans `accounts.js` (l.15 et l.39), le mot de passe en clair (`password`) est passé à la fonction `log()` lors de l'inscription et de la connexion. `logger.js` (l.12) écrit ensuite ces lignes directement dans `logs/app.log` et sur la console du serveur.
- **Preuve reproductible :** J'ai testé une tentative de connexion :

```
curl -s -X POST http://localhost:3000/api/login \
  -H "content-type: application/json" -d '{"email":"audit@test.fr","password":"MotDePasse123!"}' \
  | grep "MotDePasse123!" support/logs/app.log
```

Le mot de passe apparaît en clair dans le fichier de log.

- **Impact :** Toute personne ayant accès aux logs (développeurs, support, sauvegardes) récupère les mots de passe des utilisateurs sans aucun effort.

- **Score CVSS 3.1 :** 6.8 Moyen ([CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N](#))
- **OWASP / CWE :** A09:2021 – CWE-532.

2. Lien entre failles de sécurité et RGPD

A. Les failles constituant un risque de violation de données (Art. 4.12)

Au sens du RGPD, une violation de données est un incident entraînant la perte, l'altération ou la divulgation non autorisée de données personnelles.

- **Atteinte à la confidentialité :**
 - Le jeton admin statique (**SEC-02**) et l'absence de cloisonnement (**SEC-05**) permettent de vider l'ensemble des questionnaires médicaux et profils en une seule requête.
 - Les mots de passe en clair dans les logs (**SEC-06**) et les hashes exposés (**SEC-04**) permettent de compromettre n'importe quel compte par rebond.
- **Atteinte à l'intégrité et disponibilité :**
 - L'injection RCE (**SEC-01**) permet à un attaquant de modifier des dossiers médicaux ou d'effacer le fichier `rk.json` sur le serveur.

B. Les 3 scénarios expliquant la fuite signalée sur le forum

En observant les données et le code, j'ai identifié 3 scénarios très concrets expliquant comment les données ont pu atterrir sur le forum :

- ### 1. Scénario 1 (Le plus probable) — Appel de l'export assureur avec le jeton fuité :

Le fichier `.gitignore` ne bloque que `node_modules` et `logs`. Le fichier `db/seed.js` était donc poussé sur GitHub avec le calcul du token admin (`MS4xLjE3MDk4MDAwMDAwMDA=`). Un attaquant a juste eu à appeler : `GET /api/export/s/insurer?token=MS4xLjE3MDk4MDAwMDAwMDA=` Cette route génère **exactement un fichier JSON unique contenant les 64 salariés et leurs 134 questionnaires de santé**, prêt à être publié sur un forum.

- ## 2. Scénario 2 — Compromission du compte RH via les identifiants du README :

Le mot de passe du compte RH (`AcmeRh2024`) est public dans le README. Comme `auth.js` (l.34) donne accès à l'export assureur aux RH, n'importe qui connecté en RH a pu télécharger les données de toutes les entreprises.

- ### 3. Scénario 3 – Exploitation de l'injection JavaScript :

Un utilisateur lambda a utilisé le paramètre `filter` sur `/api/users` pour lire directement le fichier `db/wellwork.json` sur le disque du serveur.

3. Notification de violation de données (Articles 33 et 34)

A. Analyse juridique de l'obligation

- **Notification à la CNIL (Article 33) :** Obligatoire dans les 72 heures, car la fuite concerne des données de santé nominatives et engendre un risque réel pour les personnes.
- **Notification aux personnes (Article 34) :** Obligatoire dans les meilleurs délais, car la fuite présente un **risque élevé** de discrimination, de chantage ou d'usurpation d'identité pour les salariés.

B. Formulaire de notification CNIL

- **Organisme déclarant :** Start-up WellWork SAS, 12 rue de l'Innovation, 75002 Paris.
- **Contact DPO :** Me Antoine Dupont, Délégué à la Protection des Données | `dpo@wellwork.example` | 01 40 00 00 00.
- **Date et heure de constat :** 05/03/2024 à 19h13.

- **Nature de la violation** : Perte de confidentialité suite à une exfiltration de données.
- **Description factuelle** : Découverte sur un forum public d'un fichier JSON contenant les données de notre base de test. L'accès non autorisé a été rendu possible par la compromission d'un jeton d'accès d'administration associé à une route d'exportation non cloisonnée (`/api/exports/insurer`).
- **Données concernées** :
 - Données d'identité : nom, prénom, email professionnel, date de naissance, employeur (64 salariés).
 - Données de connexion : empreintes de mots de passe (SHA-256).
 - Données de santé (Art. 9) : 134 questionnaires médicaux (stress, antécédents, traitements, biométrie).
- **Personnes concernées** : 64 salariés d'entreprises clientes.
- **Conséquences probables** : Atteinte à la vie privée, rupture du secret médical, risque de discrimination par les employeurs, phishing ciblé.
- **Mesures d'urgence déjà prises** :
 1. Révocation immédiate du jeton admin fuité et suppression de toutes les sessions actives.
 2. Coupure préventive de la route `/api/exports/insurer`.
 3. Neutralisation de la fonction `new Function` sur le filtrage des utilisateurs.
 4. Réinitialisation globale forcée des mots de passe des 64 utilisateurs.
- **Mesures programmées** : Refonte de l'authentification avec jetons JWT signés et expirables, migration vers Argon2id, cloisonnement strict par entreprise.
- **Information des salariés (Art. 34)** : Déployée sous 24h par email individuel.

C. Message envoyé aux salariés concernés

Objet : Information importante concernant la sécurité de votre compte WellWork

Bonjour [Prénom],

Nous vous écrivons pour vous informer en toute transparence d'un incident de sécurité survenu sur la plateforme WellWork, ayant pu exposer certaines de vos données personnelles.

Que s'est-il passé ?

Un fichier contenant des données issues de notre plateforme a été détecté sur un forum en ligne. Dès que nous en avons été informés, nos équipes ont immédiatement bloqué l'accès en cause, sécurisé nos serveurs et révoqué l'ensemble des accès non autorisés.

Quelles sont les données concernées ?

Les informations suivantes associées à votre profil utilisateur ont pu être consultées :

- Vos informations professionnelles : nom, prénom, adresse email et entreprise de rattachement ;
- L'empreinte de votre mot de passe (celui-ci n'était pas stocké en clair, mais fait l'objet de mesures de précaution renforcées) ;
- Les réponses saisies dans vos questionnaires de bien-être (niveau de stress, habitudes de vie et antécédents déclarés).

Ce que nous avons fait immédiatement :

- Nous avons coupé la route d'exportation des données et révoqué les accès concernés ;
- Nous avons réinitialisé votre mot de passe par précaution : un lien sécurisé vous sera envoyé par email pour en définir un nouveau lors de votre prochaine connexion ;
- Nous avons officiellement notifié cet incident à la CNIL (Commission Nationale de l'Informatique et des Libertés) ;
- Une enquête technique approfondie a été ouverte pour sécuriser définitivement notre plateforme.

Ce que nous vous recommandons de faire :

1. Changer vos mots de passe identiques : Si vous utilisiez ce même mot de passe sur d'autres services (boîte mail personnelle, banque, réseaux sociaux), nous vous conseillons vivement de le modifier immédiatement sur ces autres sites.
2. Rester vigilant face aux messages suspects : Méfiez-vous des courriels, SMS ou appels inhabituels prétendant venir de votre entreprise, de votre mutuelle ou de services de santé qui vous demanderaient des identifiants ou des coordonnées bancaires.

Nous regrettons sincèrement cet incident et mettons tout en œuvre pour renforcer la sécurité de vos informations.

Notre Délégué à la Protection des Données (DPO) se tient à votre entière disposition à l'adresse suivante : dpo@wellwork.example.

L'équipe WellWork

PARTIE C — Remédiation

1. Plan de remédiation priorisé

Le plan ci-dessous organise la remédiation en 3 horizons de temps, en reliant chaque action opérationnelle aux constats techniques (SEC-xx) et réglementaires (NC-xx) :

Réf Action	Action opérationnelle	Horizon	Constats traités	Fichiers modifiés	Responsable
ACT-01	Révocation du jeton admin permanent et purge des sessions	Immédiat (H+2)	SEC-02, SEC-05	db/wellwork.json , db/seed.js	DevOps / Lead Dev
ACT-02	Suppression des mots de passe dans les logs et purge de app.log	Immédiat (H+4)	SEC-06	src/routes/account.s.js , logs/app.log	Dev Backend
ACT-03	Coupure préventive de la route d'export assureur brut	Immédiat (H+4)	NC-01, SEC-05	src/routes/data.js	Lead Dev
ACT-04	Révocation des mots de passe faibles et nettoyage du README	Immédiat (H+12)	SEC-03, SEC-04	README.md , db/wellwork.json	Lead Dev
ACT-05	Suppression de new Function et filtrage strict par whitelist	Court terme (Sprint 1)	SEC-01	src/db.js , src/routes/data.js	Dev Backend
ACT-06	Refonte de l'authentification (jetons signés avec expiration)	Court terme (Sprint 1)	SEC-02	src/auth.js	Dev Backend
ACT-07	Cloisonnement multi-tenant strict et retrait du rôle RH d'admin	Court terme (Sprint 1)	SEC-05, NC-02	src/auth.js , src/routes/data.js	Dev Backend
ACT-08	Masquage systématique de pas swordHash dans les sorties API	Court terme (Sprint 1)	SEC-04	src/routes/account.s.js , src/routes/data.js	Dev Backend
ACT-09	Hachage robuste Argon2id et rate-limiting anti-bruteforce	Court terme (Sprint 2)	SEC-03, SEC-04	src/db.js , src/routes/accounts.js	Dev Backend
ACT-10	Vrai droit à l'effacement et case de consentement explicite	Court terme (Sprint 2)	NC-04, NC-05	src/routes/account.s.js , src/routes/data.js	Dev Fullstack / DPO
ACT-11	Minimisation du questionnaire (suppression des antécédents)	Structurel (Sprint 3)	NC-03	src/routes/data.js , public/index.html	Product Owner / DPO
ACT-12	Migration vers une base PostgreSQL sécurisée chez un hébergeur HDS	Structurel (Sprint 3)	SEC-01, NC-02	src/db.js , infrastructure	Architecte / DevOps
ACT-13	Intégration DevSecOps en CI/CD (scans de secrets GitGuardian et SAST)	Structurel (Sprint 3)	SEC-01, SEC-02, SEC-06	.github/workflows , .gitignore	DevOps / RSSI

2. Correctifs appliqués sur la branche dédiée (Fix-branch)

Sur la branche de remédiation Git (Fix-branch), j'ai appliqué et testé les 5 correctifs demandés par l'énoncé (3 sécurité et 2 RGPD), avec un commit dédié pour chacun :

1. Commit 1 (Sécurité) — Neutralisation de la RCE dans `db.js` (SEC-01 / ACT-05) :

- *Modification* : Remplacement de `new Function` par un filtre direct sur des propriétés autorisées (`company`, `role`).
- *Test de non-régression* : L'envoi d'une expression JavaScript complexe ou d'un appel système retourne une erreur HTTP 400 Bad Request et n'exécute aucun code.

2. Commit 2 (Sécurité) — Suppression des mots de passe dans les logs (SEC-06 / ACT-02) :

- *Modification* : Dans `accounts.js` (l.15 et 39), passage de `{ email, company }` à `logger.log` au lieu d'inclure `password`.
- *Test de non-régression* : Vérification dans les tests qu'après un appel sur `/api/login`, le fichier `app.log` ne contient plus jamais la clé `password`.

3. Commit 3 (Sécurité) — Masquage systématique de `passwordHash` (SEC-04 / ACT-08) :

- *Modification* : Ajout d'une fonction `sanitizeUser(user)` qui supprime `passwordHash` avant toute réponse JSON.
- *Test de non-régression* : Les routes `/api/me`, `/api/users` et `/api/login` ne renvoient plus la clé `passwordHash` dans leurs réponses.

4. Commit 4 (RGPD) — Suppression du consentement forcé (NC-05 / ACT-10) :

- *Modification* : Dans `POST /api/register`, initialisation de `marketingOptIn` à `false` par défaut et suppression de l'insertion automatique de `thirdParty: true` dans la table `consents`.
- *Test de non-régression* : Un compte créé sans cocher de case a bien `marketingOptIn: false` et aucun consentement tiers n'est créé.

5. Commit 5 (RGPD) — Cloisonnement multi-tenant et restriction du rôle RH (NC-02 / ACT-07) :

- *Modification* : Dans `auth.js` (l.34), restriction de `requireAdmin` au seul rôle `admin`. Filtrage obligatoire côté serveur sur `row.company === req.user.company` pour les requêtes d'annuaire des RH.
- *Test de non-régression* : Le compte RH d'ACME ne reçoit que les salariés d'ACME et est rejeté en HTTP 403 Forbidden sur `/api/exports/insurer`.

3. Constats résiduels acceptés

Certaines modifications lourdes demandent du temps pour ne pas casser la production. Les 3 risques résiduels suivants sont acceptés de manière provisoire avec des mesures compensatoires :

• Constat résiduel n°1 : Délai de migration vers PostgreSQL (SGBD relationnel)

- *Justification* : Réécrire la couche de données et migrer les données existantes demande environ 6 semaines de développement et de tests.
- *Mesures compensatoires* : Chiffrement du disque hébergeant `wellwork.json`, permissions d'accès au fichier réduites (`chmod 600`) et surveillance des accès disque.
- *Responsable de l'acceptation* : **Directeur Technique (CTO)**.

• Constat résiduel n°2 : Période transitoire avant minimisation du questionnaire de santé

- *Justification* : Supprimer définitivement les champs d'antécédents demande de valider les nouveaux questionnaires avec les coaches et les entreprises clientes.
- *Mesures compensatoires* : Masquage complet de ces champs pour les profils publics et interdiction totale de tout export vers l'extérieur.
- *Responsable de l'acceptation* : **Product Owner et DPO**.

• Constat résiduel n°3 : Mise en place progressive des scans automatisés en CI/CD

- *Justification* : Paramétrer les outils SAST et former l'équipe va prendre quelques semaines.
- *Mesures compensatoires* : Revue de code manuelle obligatoire par un développeur senior avant chaque mise en production.

- Responsable de l'acceptation : **RSSI**.
-

PARTIE D — Méthodologie agile

1. Équipe de remédiation

Pour mener à bien ces correctifs avant l'audit du grand compte, nous mettons en place une équipe agile de 6 personnes pour 2 mois :

- **1 Product Owner (PO)** : Gère le backlog et priorise les actions avec les clients.
 - **1 Délégué à la Protection des Données (DPO)** : Valide la conformité juridique et les formulaires.
 - **1 RSSI / Lead Sécurité** : Définit l'architecture sécurisée et valide les tests de non-régression.
 - **2 Développeurs Fullstack** : Développent les correctifs backend et frontend.
 - **1 DevOps / SecOps** : Configure la CI/CD, sécurise le serveur et prépare la migration PostgreSQL.
-

2. Backlog : User Stories et Abuser Stories

Epic 1 : Authentification et gestion de session

- **User Story (US-01)** : *En tant que salarié, je veux me connecter avec un mot de passe sécurisé et un jeton temporaire pour que mon compte soit protégé.*
 - Critères d'acceptation : Jeton signé HMAC-SHA256 expirant au bout d'une heure ; mot de passe haché avec Argon2id.
- **Abuser Story (AS-01)** : *En tant qu'attaquant, je veux tester des milliers de mots de passe sur la page de login pour trouver des accès.*
 - Critères d'acceptation : Blocage après 5 échecs en 15 minutes (HTTP 429) et message d'erreur générique ne distinguant pas email et mot de passe.

Epic 2 : Confidentialité et cloisonnement des données médicales

- **User Story (US-02)** : *En tant que salarié, je veux être sûr que mon employeur et mes collègues ne peuvent pas lire mes questionnaires de santé.*
 - Critères d'acceptation : Seul le salarié peut accéder à ses questionnaires via `/api/users/:id` ; le RH reçoit une erreur 403 s'il tente d'y accéder.
- **Abuser Story (AS-02)** : *En tant que RH d'une entreprise, je veux exporter les questionnaires de santé des salariés d'entreprises concurrentes.*
 - Critères d'acceptation : La route d'exportation `/api/exports/insurer` est interdite aux RH ; l'annuaire filtre strictement sur l'entreprise du RH.

Epic 3 : Droits RGPD et consentement

- **User Story (US-03)** : *En tant qu'utilisateur, je veux pouvoir supprimer définitivement mon compte et toutes mes données médicales.*
 - Critères d'acceptation : L'appel `DELETE /api/me` supprime physiquement la ligne utilisateur et purge l'ensemble des questionnaires associés.
-

3. Méthode d'estimation et de priorisation

Pour prioriser les tâches du backlog, nous utilisons une formule simple inspirée de la méthode WSJF :

Priorité = (Score de risque CVSS ou RGPD + Valeur de conformité) / Effort technique en points

Les failles critiques avec exploit public (la RCE `new Function`, le jeton admin codé en dur et la fuite des logs) sont traitées en priorité absolue dès le premier sprint.

4. Découpage en 3 sprints de 2 semaines

- **Sprint 1 — Confinement d'urgence et sécurité applicative :**

- *Objectif* : Éliminer les failles exploitables à distance et rétablir le cloisonnement.
- *Tâches* : Remplacement de `new Function` (ACT-05), mise en place de jetons signés (ACT-06), dissociation du rôle RH et cloisonnement tenant (ACT-07), nettoyage des logs (ACT-02), masquage de `passwordHash` (ACT-08).

- **Sprint 2 — Conformité RGPD et effectivité des droits :**

- *Objectif* : Répondre aux exigences CNIL et respecter les droits des utilisateurs.
- *Tâches* : Case à cocher pour le consentement santé (ACT-10), vraie purge physique sur `DELETE /api/me` (ACT-10), intégration du rate-limiting et hachage Argon2id (ACT-09), ajout des mentions d'information légales sur l'interface (NC-07).

- **Sprint 3 — Résilience structurelle et DevSecOps :**

- *Objectif* : Assurer la pérennité technique pour l'audit du grand compte.
 - *Tâches* : Réduction des champs du questionnaire (ACT-11), migration vers PostgreSQL sécurisé (ACT-12), mise en place de GitGuardian et scans SAST dans la CI/CD (ACT-13).
-

5. Definition of Done (DoD)

Pour qu'un ticket soit validé ("Done"), il doit respecter les critères suivants :

1. Code relu et approuvé par un pair et le Lead Développeur ;
 2. Tests unitaires et tests de non-régression passés avec succès ;
 3. Aucune donnée sensible (mot de passe, jeton, hash) présente dans les logs ou réponses API ;
 4. Validation formelle par le DPO sur les aspects consentement et minimisation ;
 5. Analyse statique (SAST) et scan de secrets sans aucune alerte haute ou critique.
-

6. Cérémonies agiles et gestion des risques

- **Rôle du DPO et du RSSI :**

- Présence obligatoire au **Sprint Planning** pour valider les critères de conformité et de sécurité de chaque ticket ;
- Présence à la **Sprint Review** pour valider la fermeture officielle des constats d'audit ;
- Intervention aux **Daily Standups** en cas d'alerte de sécurité.

- **Indicateurs de suivi :**

- Burndown chart et vélocité de l'équipe ;
- Nombre de failles de sécurité ouvertes par criticité ;
- Couverture des tests de non-régression ;
- Pourcentage de conformité au registre de l'article 30.

- **Gestion des risques :**

- *Risque de régression métier* : Tests automatisés systématiques avant toute mise en ligne.
 - *Risque de perte du grand compte* : Présentation intermédiaire des résultats à la fin du Sprint 1 pour prouver que les failles critiques ont été corrigées.
-

Annexe — Transparence sur l'usage de l'IA (Section 4 de l'énoncé)

1. Outils utilisés : Gemini 3.8 Flash et le Mode IA de Chrome

2. Principales requêtes formulées :

- "Quels sont les articles du RGPD applicables à une plateforme de santé au travail ?"
- "Comment calculer le score CVSS v3.1 d'une injection de code dans new Function Node.js ?"
- "Donne-moi un plan type pour une notification de violation de données à la CNIL."
- "Relecture, harmonisation et mise au propre du document final en syntaxe Markdown standard"

3. Erreurs et imprécisions de l'IA détectées et corrigées :

- *Erreur 1 (Statut juridique)* : L'IA m'a d'abord proposé de mettre l'employeur et WellWork comme co-responsables sur tout. J'ai corrigé cette erreur car en droit du travail, un employeur n'a absolument pas le droit d'avoir accès aux données de santé individuelles de ses salariés.
- *Erreur 2 (Droit à l'oubli)* : L'IA affirmait qu'un soft-delete avec `deleted: true` suffisait. J'ai corrigé en constatant que les questionnaires de santé restaient en base indéfiniment et qu'une vraie purge physique était indispensable.
- *Erreur 3 (Cause de la fuite)* : L'IA a commencé par imaginer une injection SQL complexe. J'ai corrigé immédiatement car en lisant le code, il n'y a aucune base SQL dans WellWork : tout fonctionne avec un fichier JSON et la fuite venait de la route `/api/exports/insurer` appelée avec le token du script `seed.js`.
- *Erreur 4 (Champs du questionnaire)* : L'IA a inventé des champs génériques comme la tension artérielle. J'ai corrigé avec les données réelles du code (`seed.js` 1.58-64 : stress, antécédents, traitements, sommeil, poids, taille, tabac).
- *Erreur 5 (Obligation d'AIPD)* : L'IA m'a répondu au début que l'AIPD était facultative pour les petites structures de moins de 250 salariés. J'ai corrigé en appliquant les critères du CEPD (WP248) : dès qu'il y a des données de santé sur des salariés avec croisement de données, l'AIPD est obligatoire quelle que soit la taille de l'entreprise.